RISC-V 五级流水线 CPU 内核

数据通路（Data Path）与状态机（FSM）设计说明

RV32I 基础整数集 · 37 条指令 · 六种指令格式全覆盖

IF 取指 → ID 译码 → EX 执行 → MEM 访存 → WB 写回

文档要点

1. **数据通路组成结构与逐级数据流向**
   五级流水线模块清单、每级数据流公式、4 组流水线寄存器锁存内容

2. **状态机（FSM）建模与状态定义**
   状态寄存器 = 5 组触发器；状态集 {IF, ID, EX, MEM, WB} 与物理映射

3. **状态转移条件与控制逻辑**
   主推进 / 停顿 Stall / 冲刷 Flush / 复位四类转移；maindec、aludec、Hazard 输出函数

设计要点

流水线	5 级 · IF / ID / EX / MEM / WB，4 组流水线寄存器 F_D、D_E、E_M、M_W
冒险处理	数据前推（EX/MEM、MEM/WB 两级）· load-use 停顿 1 拍 · 分支与跳转冲刷 2 条
控制方式	分布式控制：译码阶段组合产生控制位，随流水线寄存器逐级传播
验证结果	39 项断言 ALL TESTS PASSED；指令集经 4 个阶段由 13 条扩展至 37 条

2026 年 10 月

1 指令子集与寄存器规范

采用 RISC-V RV32I 基础整数集（不含 FENCE / ECALL / EBREAK / CSR），共 37 条指令，覆盖 R / I / S / B / U / J 六种指令格式。32 个 32 位通用寄存器，x0 恒为 0（硬件强制，写入被丢弃）；小端字节序，仅支持字、半字、字节三种宽度；寻址模式为「基址寄存器 + 符号扩展立即数」，仅 load / store 访问内存。

类别	指令	条数	格式
R 型 寄存器-寄存器	add sub sll slt sltu xor srl sra or and	10	R
I 型 寄存器-立即数	addi slti sltiu xori ori andi slli srli srai	9	I
载入 Load	lb lh lw lbu lhu	5	I
存储 Store	sb sh sw	3	S
分支 Branch	beq bne blt bge bltu bgeu	6	B
跳转与高位立即数	jal jalr lui auipc	4	J / I / U
合计	RV32I 基础整数集（除系统指令外全部实现）	37	6 种

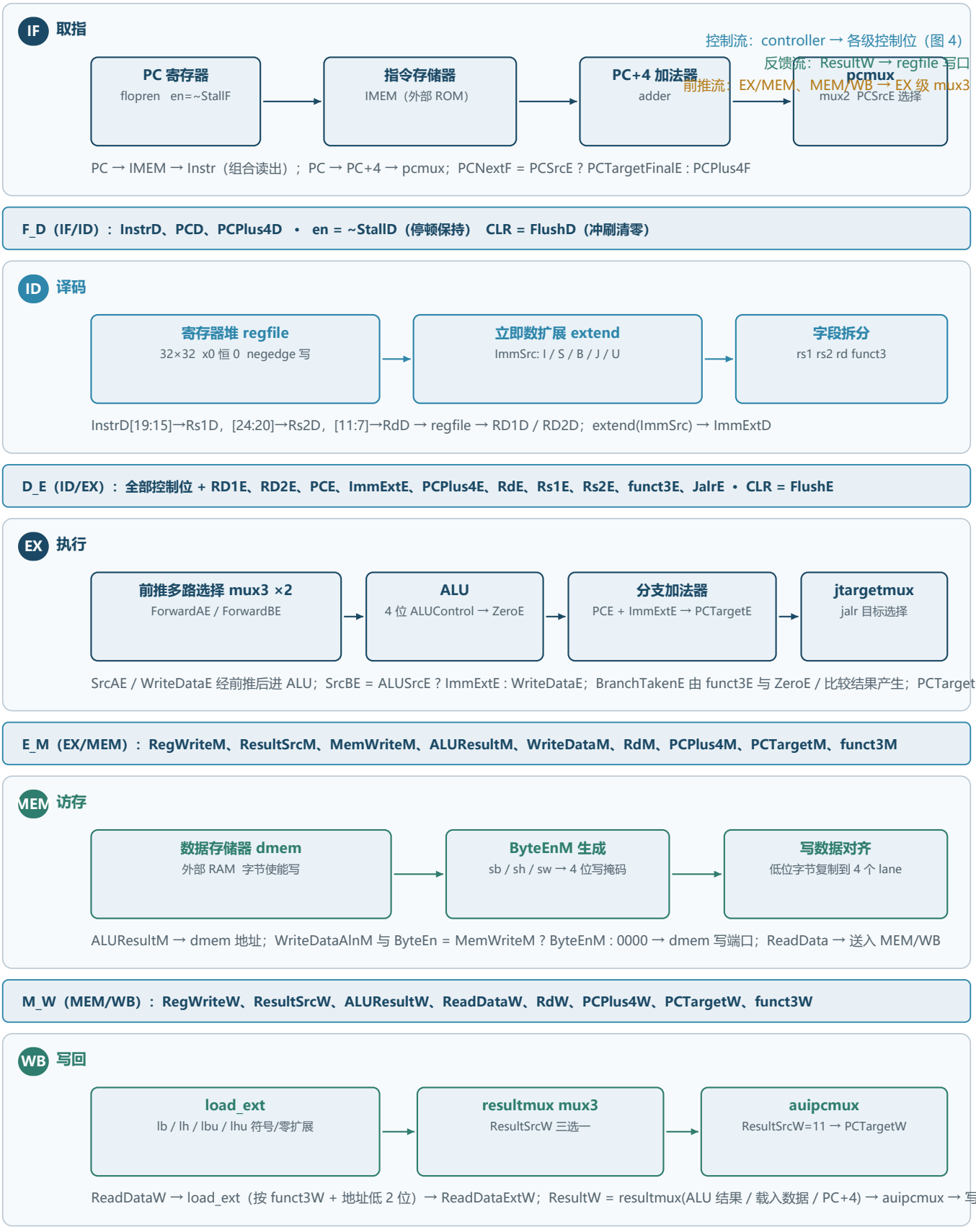
2 流水线时序（含停顿与冲刷）

	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12
I1 lw x14,0(x0)	IF	ID	EX	MEM	WB							
I2 add x15,x14,x1		IF	ID	ID*	EX	MEM	WB					
I3 sub x17,x2,x16			IF	IF*	ID	EX	MEM	WB				
I4 beq x1,x5,+12					IF	ID	EX	MEM	WB			
I5 addi（被冲刷）						IF	X					
I6 ori （被冲刷）							X					
I7 addi（分支目标）								IF	ID	EX	MEM	WB

I2 与 I3 因 lw→add 的 load-use 冒险停顿 1 拍（PC 与 IF/ID 保持，ID/EX 清零）；I4 为 taken 分支，I5、I6 在 EX 级被冲刷。

图 1 五级流水线时序与冒险插空示意（* 表示该拍为停顿/重复取指，X 表示被冲刷丢弃）

图 2　五级流水线数据通路（模块、流水线寄存器与主要数据/控制流向）



1 逐级数据流向

IF 取指级

组成模块	PC 寄存器 flopren、PC+4 加法器 adder、下址选择 pcmux(mux2)；外部指令存储器 IMEM
数据流向	PC → IMEM → Instr（组合读出，PC 变化后当拍即可取到指令）；PC → +4 → PCPlus4F；PCNextF = PCSrcE ? PCTargetFinalE : PCPlus4F
时序控制	pcreg 使能 en = ~StallF：load-use 停顿时 PC 保持不变，实现取指冻结

ID 译码级

组成模块	寄存器堆 regfile（32×32，x0 恒 0，时钟下降沿写）、立即数扩展 extend、指令字段拆分
数据流向	InstrD[19:15] / [24:20] / [11:7] 拆出 Rs1D / Rs2D / RdD；regfile 读口 → RD1D、RD2D；extend(InstrD[31:7], ImmSrc) → ImmExtD
说明	立即数格式由 ImmSrc 三位于组合逻辑中选出：000=I、001=S、010=B、011=J、100=U

EX 执行级

组成模块	两个前推 mux3（SrcA / SrcB）、立即数选择 mux2、ALU、分支加法器 adder、jalr 目标 mux2、Hazard 冒险单元
数据流向	RD1E 经前推 → SrcAE；RD2E 经前推 → WriteDataE；SrcBE = ALUSrcE ? ImmExtE : WriteDataE；ALU → ALUResultE 与 ZeroE；PCE + ImmExtE → PCTargetE
跳转目标	PCTargetFinalE = JalrE ? {ALUResultE[31:1], 1'b0} : PCTargetE —— jalr 的目标取自 rs1 + imm（ALU 结果），并清除最低位；jal 与分支则使用 PC + 立即数

MEM 访存级

组成模块	外部数据存储器 dmem、写使能生成（ByteEnM）、写数据对齐（WriteDataAlnM）
数据流向	ALUResultM → 访存地址；WriteDataAlnM 与 ByteEn 一起送 dmem 写端口；ReadData → 送入 MEM/WB 寄存器
字节对齐	sw 时 ByteEnM = 1111；sh 时 = 0011 << 地址[1]；sb 时 = 0001 << 地址[1:0]。写数据同时把低位字节 / 半字复制到 4 个 lane，由写掩码选通

WB 写回级

组成模块	load_ext 载入扩展、resultmux(mux3)、auipcmux(mux2)
数据流向	ReadDataW → load_ext（依据 funct3W 与地址低 2 位抽取字节 / 半字并做符号或零扩展）→ ReadDataExtW；resultmux 在 ALUResultW、ReadDataExtW、PCPlus4W 之间按 ResultSrcW 三选一 → ResultW_pre；ResultSrcW = 11 时 auipcmux 选 PCTargetW → ResultW → regfile 写口
写口约束	regfile 仅在 RegWriteW 有效且 RdW ≠ 0 时写入；实际写入发生在时钟下降沿，避免同一时钟边沿「先读后写」的竞争

1 四组流水线寄存器 + PC 寄存器 = 五组状态寄存器

本设计要求说明状态机。需要指出：本 CPU 的控制单元并非「一条指令一个显式状态」的多周期控制器，而是分布式组合控制；其状态由五组触发器共同承载——四条指令可同时在流水线中处于不同阶段，机器状态就是这五组寄存器的内容快照。下表给出各组寄存器锁存的信息与其对应的逻辑状态。

状态寄存器	对应逻辑状态	锁存内容	例外控制
pcreg (flopren)	S_IF	PCF： 当前取指地址	en = ~StallF： 停顿时保持
F_D (IF/ID)	S_ID	InstrD（32 位指令）、PCD、 PCPlus4D	en = ~StallD 保持； CLR = FlushD 清零
D_E (ID/EX)	S_EX	全部控制位 + RD1E/RD2E、 PCE/ImmExtE/PCPlus4E、RdE、 Rs1E/Rs2E/funct3E/JalrE	CLR = FlushE 清零（气泡）
E_M (EX/MEM)	S_MEM	RegWriteM/ResultSrcM/ MemWriteM/ALUResultM/ WriteDataM/RdM、PCPlus4M/ PCTargetM/funct3M	无 （仅 reset 清零）
M_W (MEM/WB)	S_WB	RegWriteW/ResultSrcW/ ALUResultW/ReadDataW/ RdW/PCPlus4W/PCTargetW/ funct3W	无 （仅 reset 清零）

注意：控制位不集中于一个独立的控制状态寄存器，而是随指令逐级「漂流」——译码级产生的控制位每组流水线寄存器各锁存一份，因此同一条指令在 ID 级被译码一次，其控制效果却在 EX / MEM / WB 三级各自使用对应的那份副本。

2 控制位在主译码器中的编码

maindec 以 7 位 opcode 为输入，输出 13 位控制向量；其中 ResultSrc 与 ALUOp 交给下游逻辑做二次选择：ResultSrc 决定写回数据来源，ALUOp 与 funct3 / funct7 一起决定 ALU 运算。

```
assign {RegWrite, ImmSrc, ALUSrc, MemWrite,
        ResultSrc, Branch, ALUOp, Jump, Jalr} = controls;
```

1 状态定义

把机器状态定义为「当前流水线中每条指令所处的阶段」。设状态集 S = { S_IF, S_ID, S_EX, S_MEM, S_WB }，一条指令在其生命周期中依次经历这五个状态，每周期前进一个状态；同一时刻流水线中最多有 5 条指令分别处于这 5 个状态。状态由 S 中五个状态寄存器组的取值承载，输出（控制信号）仅由当前状态中的指令字段决定，因此本机是一个 Moore 型有限状态机。

状态	含义	承载寄存器	该状态完成的动作	输出函数
S_IF	取指	pcreg	PC 送出地址，从 IMEM 取出指令	PCNextF 选择
S_ID	译码	F_D	拆字段、读寄存器堆、扩展立即数	maindec / aludec
S_EX	执行	D_E	前推、ALU 运算、分支判据、跳转目标	ForwardA/B、PCSrcE
S_MEM	访存	E_M	读写数据存储器、字节使能与对齐	ByteEn、MemWrite
S_WB	写回	M_W	载入扩展、写回寄存器堆	ResultSrc、RegWriteW

2 状态转移图

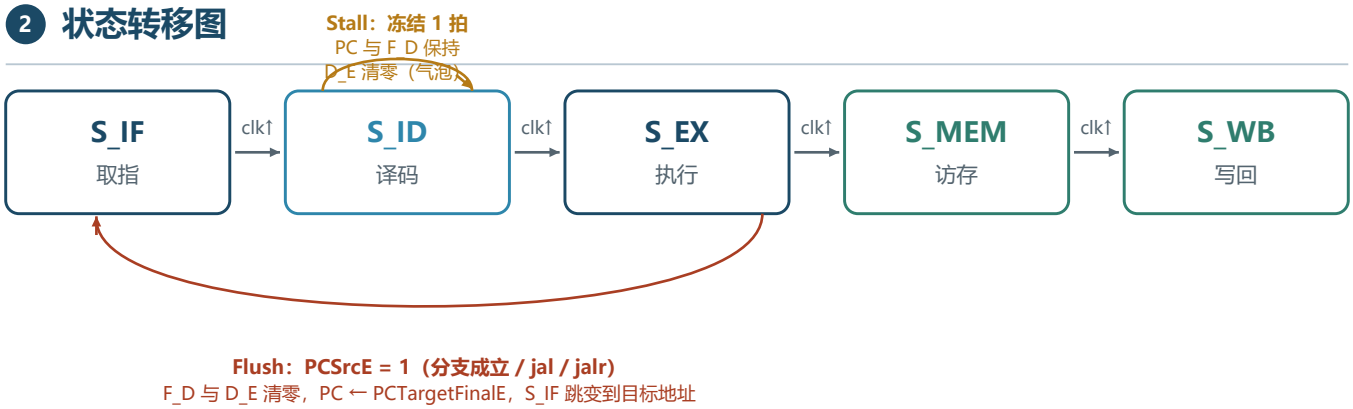


图 3 状态转移图：实线 clk↑ 为主推进；琥珀色自环为 load-use 停顿；红色弧为分支/跳转冲刷；reset 使所有状态寄存器清零、PC = 0（未单独画出）。

1 四类状态转移

转移类型	触发条件	状态寄存器的动作	结果
复位 reset	reset = 1	PC、F_D、D_E、 E_M、M_W 全部清零 pcreg ← PCNextF	PC = 0， 从第一条指令重新开始
主推进	每个 clk 上升沿， StallF=0 且 CLR=0	F_D ← IF 结果 D_E ← ID、E_M ← EX M_W ← MEM	5 条指令同时 前进一级 单周期吞吐
停顿 Stall	lwStall = 1 （load-use 冒险）	pcreg 保持（en=0） F_D 保持（en=0） D_E 清零（CLR=1）	IF、ID 两级冻结 1 拍 EX 级插入气泡 共停顿 1 个周期
冲刷 Flush	PCSrcE = 1 （分支成立 / jal / jalr）	F_D 清零（CLR=1） D_E 清零（CLR=1） pcreg ← PCTargetFinalE	两条错误指令被丢弃 PC 重定向到目标地址

2 转移条件与输出函数的实现

冒险检测与停顿、冲刷条件（Hazard 单元，纯组合逻辑）

```
lwStall = ResultSrcE[0] & ((Rs1D == RdE) | (Rs2D == RdE));  // load-use 冒险
StallF  = lwStall;   StallD = lwStall;   FlushE = lwStall | PCSrcE;
FlushD  = PCSrcE;
```

数据前推选择（EX 级多路选择，优先取最近一级结果）

```
ForwardA/B = 10  if (RegWriteM && RdM != 0 && RdM == Rs1E/Rs2E)  // EX/MEM 前推
           = 01  else if (RegWriteW && RdW != 0 && RdW == Rs1E/Rs2E) // MEM/WB 前推
           = 00  else                                  // 用寄存器堆读出的值
```

分支判据与 PC 选择（EX 级，funct3E 区分六种分支）

```
BranchTakenE = BranchE & ( (funct3E==000 & ZeroE) | (funct3E==001 & ~ZeroE)
                         | (funct3E==100 & LessE) | (funct3E==101 & ~LessE)
                         | (funct3E==110 & LessUE)| (funct3E==111 & ~LessUE) );
PCSrcE = BranchTakenE | JumpE;
PCTargetFinalE = JalrE ? {ALUResultE[31:1], 1'b0} : PCTargetE;
```

设计要点：分支与跳转的判据全部在 EX 级产生，因此只需冲刷 IF 与 ID 两级——这正是把分支比较器提前到 EX 级、并配以 EX/MEM、MEM/WB 两级前推的直接收益：分支代价从「等操作数写回」的 3 拍降为 2 拍。

1 主译码器 maindec 真值表

控制向量共 13 位，按 {RegWrite, ImmSrc[2:0], ALUSrc, MemWrite, ResultSrc[1:0], Branch, ALUOp[1:0], Jump, Jalr} 排列，由 opcode 唯一确定；未定义 opcode 一律输出全 0（等效 nop）。

指令	opcode	RegWr	ImmSrc	ALUSrc	MemWr	ResultSrc	Branch	ALUOp	Jump	Jalr
lw	0000011	1	000 I	1	0	01 载入	0	00	0	0
sw	0100011	0	001 S	1	1	00 ALU	0	00	0	0
R 型	0110011	1	000 I	0	0	00 ALU	0	10	0	0
分支	1100011	0	010 B	0	0	00 ALU	1	01	0	0
I 型 ALU	0010011	1	000 I	1	0	00 ALU	0	10	0	0
jal	1101111	1	011 J	0	0	10 PC+4	0	00	1	0
lui	0110111	1	100 U	1	0	00 ALU	0	11	0	0
auipc	0010111	1	100 U	0	0	11 PC+imm	0	00	0	0
jalr	1100111	1	000 I	1	0	10 PC+4	0	00	1	1
未定义	其它	0	000	0	0	00	0	00	0	0

2 ALU 译码器 aludec 与运算编码

ALUOp	选择依据	ALUControl	运算
00	lw / sw / auipc / jalr：地址加法	0000	add（加）
01	分支指令：减法判零	0001	sub（减）
11	lui：立即数直通	1010	SrcB 直通
10	funct3 = 000 且 funct7[5] = 1（R 型）	0001	sub
10	funct3 = 000（其余）	0000	add
10	funct3 = 001	0111	sll（逻辑左移）
10	funct3 = 010	0101	slt（有符号小于）
10	funct3 = 011	0110	sltu（无符号小于）
10	funct3 = 100	0100	xor（异或）
10	funct3 = 101 且 funct7[5] = 1	1001	sra（算术右移）
10	funct3 = 101 且 funct7[5] = 0	1000	srl（逻辑右移）
10	funct3 = 110	0011	or（或）
10	funct3 = 111	0010	and（与）

1 数据前推通路

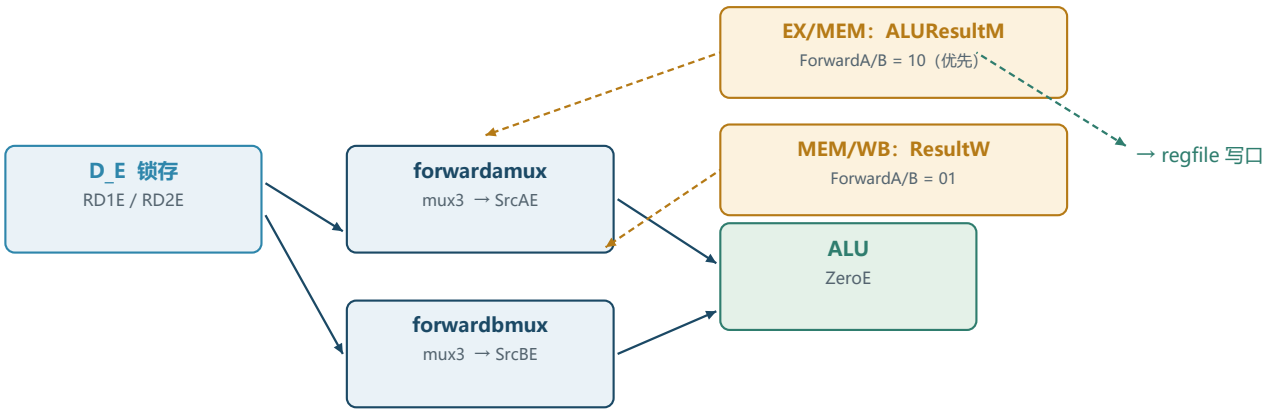


图 4 前推通路：EX/MEM 与 MEM/WB 两级结果回送 EX 级，使紧邻依赖无需等待写回；只有「load 后立即使用」一种情形才需要停顿。

2 停顿与冲刷的状态动作



图 5 一个「load-use 停顿 + 分支冲刷」连续发生的状态序列（对应图 1 的 I2 / I4 段）

3 验证结论

验证项	结论
流水线骨架	五级寄存器链、PC 递增、寄存器写回正确（x1–x4、x7 等断言通过）
数据前推	EX/MEM 与 MEM/WB 两级前推均生效，紧邻 ALU 依赖无需停顿
load-use 停顿	lw 后紧接使用其结果的指令停顿 1 拍，PC 与 IF/ID 保持、ID/EX 插入气泡
分支与跳转	六种分支的 taken / not-taken 组合、jal 链接、jalr 返回与重定向全部正确
指令集扩展	13 条 → 23 条 → 31 条 → 37 条四阶段，每阶段回归测试无退化
总量	39 项断言 ALL TESTS PASSED，RV32I 基础整数集全部实现

后续工作：补 soc_top（CPU + ROM/RAM + MMIO 译码）以便逻辑综合，加时钟约束评估最高工作频率，并完成 FPGA 上板流水灯 Demo。